

DuckDB from the Command Line

Windows: install DuckDB and run your first queries

OMIS 105 - Fall 2026

Examples use **DuckDB 1.5.5**. Your installation may show a newer version.

Reviewed October 2, 2026. Start here, then follow the slides in order.

What You Will Learn

1. Install the standalone DuckDB command-line program.
2. Fix “command not found” or “not recognized.”
3. Open the tutorial folder and query its CSV file.
4. Save a database, change output format, and run a SQL file.
5. Diagnose file, permission, and session errors.

Keep this guide and `data/students.csv` together. You will create `report.sql` during the lesson.

Follow the main lesson first. Use **Help Only If Needed** at the end only when a step fails. You do not have to memorize troubleshooting.

Four Words Used in This Guide

Word	Meaning here
Terminal / shell	The window where you type commands to your computer.
CLI	“Command-line interface”: the DuckDB program used in this lesson.
Path	A file or folder's location on your computer.
PATH	The list of folders searched when you type a program name.

Enter and **Return** mean the same key in these instructions.

After each command, wait for the result before typing the next one.

Two Prompts, Two Kinds of Commands

Where you are	Examples to enter there
Terminal shell	<code>duckdb --version</code> , <code>duckdb</code> , <code>cd</code>
DuckDB prompt (<code>D</code>)	<code>SELECT 42;</code> , <code>.tables</code> , <code>.quit</code>

The prompt is a label: **do not type** `D` , `$` , `%` , or `PS>` .

Copy command blocks exactly. Blocks labeled “output” show results; do not type those results.

If you see `D` , enter `.quit` before a terminal command.

If you see `>>>` , you are in Python: enter `exit()` first.

Before Installing

The **CLI** is the DuckDB program you run in a terminal. Having DuckDB work in Python or Marimo does not mean this program is installed.

In your terminal, run:

```
duckdb --version
```

If it prints a version, skip installation and go to **Open the Tutorial Folder**.

If it fails, follow the installation steps next. Choose only one method.

No Python environment, database server, or DuckDB account is needed.

Windows: Choose Your Shell

For a first attempt, use **Command Prompt**: press the Windows key, type `cmd`, and press Enter. If you already use PowerShell, follow the blocks labeled **PowerShell** instead.

Commands labeled PowerShell and Command Prompt are different. Do not use WSL, Git Bash, or a Python prompt for these steps.

Try `duckdb --version`. If not recognized, choose WinGet or the manual route below. Windows requires the matching [Microsoft Visual C++ Redistributable](#).

Windows Option A: WinGet

In either shell, one command at a time:

```
winget --version  
winget install --id DuckDB.cli --exact
```

Wait for **Successfully installed**. If it fails, read the error first.

WinGet is supplied through Microsoft's **App Installer**; it may be missing or restricted on managed PCs. Use the manual route if permitted.

Close and reopen the entire terminal application, then try

`duckdb --version`. If it works, skip Option B and go to **Windows: Open the Tutorial Folder**. Otherwise use the help section at the end.

Windows Option B: Download and Extract

1. At duckdb.org/install, select **Windows / CLI / current stable**.
2. In **Settings > System > About**, check **System type**.
Use the **amd64** zip for x64 Intel/AMD PCs or **arm64** for Arm PCs.
3. Right-click the downloaded zip and select **Extract All**.
4. In File Explorer's address bar, enter `%USERPROFILE%`. Create a folder named **duckdb-cli** and move **duckdb.exe** into it.

Do not run from inside the zip. Enable **View > Show > File name extensions** to check the filename. A user-owned folder avoids needing permission to create `C:\duckdb`.

Windows: Test Before Editing PATH

PowerShell:

```
Test-Path "$env:USERPROFILE\duckdb-cli\duckdb.exe"  
& "$env:USERPROFILE\duckdb-cli\duckdb.exe" --version
```

Command Prompt:

```
dir "%USERPROFILE%\duckdb-cli\duckdb.exe"  
"%USERPROFILE%\duckdb-cli\duckdb.exe" --version
```

If the file is absent, fix its location first. PowerShell needs `&` to execute a quoted path. A version number confirms the executable works.

Windows: Make duckdb Work by Name

For the **manual route**, add the folder containing `duckdb.exe` to PATH:

1. Start menu: search **Edit environment variables for your account**.
2. Under **User variables**, edit **Path**; create it if absent.
3. Add a **new entry**: the full path to your `duckdb-cli` folder.
Copy it from File Explorer. Do not include `duckdb.exe` or quote marks.
4. Keep existing entries. Do not replace the entire Path value.
5. Close and reopen Windows Terminal, PowerShell, Command Prompt, and VS Code as applicable. Then run `duckdb --version`.

If editing PATH is blocked, use the full-path command from the previous page. `%USERPROFILE%` and `$env:USERPROFILE` mean your home folder; copy them exactly when shown in commands.

Windows: Open the Tutorial Folder

Extract the **whole tutorial folder**, including `data/students.csv`.

In File Explorer, open that folder, type **cmd** in its address bar, and press Enter. A Command Prompt opens at the correct location. Run:

```
cd  
dir data\students.csv
```

If using PowerShell instead, use **Copy as path** on the folder and enter

```
Set-Location "the copied full folder path", then Get-Location.
```

The quoted path is a placeholder: replace it with your actual folder.

Windows: Paths Differ Between Shells

To navigate manually, replace the example path with your real location:

Command Prompt:

```
cd /d "C:\Users\maria\Documents\DuckDB_from_Command_Line"
```

PowerShell:

```
Set-Location "C:\Users\maria\Documents\DuckDB_from_Command_Line"
```

`/d` lets Command Prompt change drives too. `%USERPROFILE%` is CMD syntax; `$env:USERPROFILE` is PowerShell syntax. Bare `cd` reports the folder in CMD but changes to the home folder in PowerShell.

Checkpoint: Does DuckDB Run?

In your terminal:

```
duckdb --version  
duckdb -c "SELECT 6 * 7 AS answer;"
```

Expect a version and the answer **42**. An illustrative version is:

```
v1.5.5 (Variegata) d8cdaa33fd
```

If typing `duckdb` still fails, see **Help Only If Needed** at the end.

It explains how to start DuckDB using the location of its program file.

Checkpoint: Can It Read the Sample?

In your terminal, in the tutorial folder:

```
duckdb -c "SELECT COUNT(*) AS students FROM read_csv('data/students.csv');"
```

Expect **8**. If this fails but the 42 check worked, installation is OK: check your current folder and the CSV filename.

The CSV should begin `id,name,major,gpa`. A file containing HTML is a saved web page, not the dataset. Download the actual file or whole repo.

Once both checkpoints pass, proceed with the SQL lesson.

Start DuckDB and Type a Query

In your terminal:

```
duckdb
```

When you see **D**, enter this SQL and press Enter/Return:

```
SELECT 6 * 7 AS answer;
```

You should see **42**. Startup text and spacing vary by version.

This session is **in-memory**: its database tables disappear on exit.

Writing a separate file is different; exported files remain on disk.

Finish or Cancel an Incomplete Command

SQL statements finish with a **semicolon** (`;`). DuckDB may show a continuation prompt if a statement, quote, or parenthesis is incomplete.

- For a missing semicolon, type `;` and press Enter.
- If you pasted the wrong text or have unmatched quotes, press **Ctrl+C** to cancel the pending input, then re-enter the complete command.
- Dot-commands such as `.quit` must start on their own fresh line. They have **no semicolon**.

A continuation prompt does not mean every problem is a missing `;`.

Query the CSV Directly

At the DuckDB prompt, run each block separately:

```
SELECT * FROM read_csv('data/students.csv');
```

```
SELECT major, ROUND(AVG(gpa), 2) AS avg_gpa  
FROM read_csv('data/students.csv')  
GROUP BY major  
ORDER BY avg_gpa DESC, major;
```

No import step is needed. This reads the CSV; it does not create a stored table named `students`. DuckDB looks for `data/` inside the folder from which you started the program.

Expected GPA Results

major	avg_gpa
Information Systems	3.83
Accounting	3.50
Finance	3.35
Marketing	3.35

Finance and Marketing tie; sorting by `major` breaks the tie consistently. DuckDB may display `3.5` rather than `3.50`; they are the same number. The default **duckbox** format may also show column types.

Create a Table for This Session

At the DuckDB prompt:

```
CREATE TABLE students AS  
SELECT * FROM read_csv('data/students.csv');
```

Then enter these **one line at a time**:

```
.tables  
.schema students
```

`.tables` lists stored tables/views, not every CSV in the folder.

If `students` already exists, reuse it; do not rerun CREATE blindly.

Change How Results Look

At the DuckDB prompt, one line at a time:

```
.mode csv  
.headers on  
SELECT id, name FROM students ORDER BY id LIMIT 2;
```

Expected CSV output:

```
id,name  
1,Alice  
2,Bob
```

Other modes: `.mode markdown`, `.mode json`, `.mode table`.

Use `.mode duckbox` to return to the default display.

Headers and Timing

At the DuckDB prompt, one line at a time:

```
.mode csv  
.headers off  
SELECT COUNT(*) FROM students;  
.headers on  
.timer on  
SELECT COUNT(*) FROM students;  
.timer off  
.mode duckbox
```

The first count is **8** without a header. The second adds a header and timing information. Timing values vary; these settings persist in the session.

Save a Database and Reopen It

First exit the in-memory session with `.quit`. **In your terminal:**

```
duckdb class_cli.db
```

At D, create the table once in this file database:

```
CREATE TABLE students AS  
SELECT * FROM read_csv('data/students.csv');
```

Enter `.quit`, then run `duckdb class_cli.db` again in the same folder.

At D, `SELECT COUNT(*) FROM students;` should still return **8**.

If the table already exists, skip CREATE and query it.

Keep Track of Your Saved Database

- A relative `.db` filename is relative to the current folder. Starting from a different folder can create a different, empty database.
- Use `.databases` at D to inspect the open database; use `.tables` to inspect its tables. Quote terminal paths containing spaces.
- A database's parent folder must already exist and be writable.
- `.open file.db` switches databases; it does **not** save/copy your previous in-memory tables into the file.
- The statements in this lesson save automatically in a file database. If your instructor has you use `BEGIN`, use `COMMIT` before exiting.

Create a Script File

In a plain-text editor, save these lines as **report.sql** in the tutorial folder. Use VS Code, Notepad (Save as type: All files), or TextEdit in plain-text mode. Check the file is not `report.sql.txt`.

```
SELECT COUNT(*) AS students FROM read_csv('data/students.csv');  
SELECT major, ROUND(AVG(gpa), 2) AS avg_gpa  
FROM read_csv('data/students.csv')  
GROUP BY major  
ORDER BY avg_gpa DESC, major;
```


Run Your Script File

After saving **report.sql**, exit DuckDB with `.quit` first.

In the terminal, from this tutorial folder:

```
duckdb -bail -f report.sql
```

Expect the row count **8**, then the four GPA results shown earlier.

`-bail` stops if an error occurs. `-f` runs the SQL file and exits.

Alternatively, while already at D, enter `.read report.sql`.

Run this command from the tutorial folder. DuckDB looks for `data/` there, even if your SQL file is saved somewhere else.

Optional Practice: Write Another SQL File

Use a plain-text editor (VS Code, Notepad, or TextEdit in plain-text mode).

Save this as **my_query.sql** in the tutorial folder:

```
SELECT major, COUNT(*) AS n
FROM read_csv('data/students.csv')
GROUP BY major
ORDER BY major;
```

Exit DuckDB first, then run `duckdb -bail -f my_query.sql` in the shell.

Check it is not `my_query.sql.txt` or rich text. In Notepad, choose

All files. Use straight quotes, not smart/curly quotes.

Quotes, Spaces, and Copy/Paste

For simple one-off SQL, double-quote the shell argument and single-quote SQL strings:

```
duckdb -c "SELECT 'Finance' AS major;"
```

SQL uses single quotes for values; double quotes identify columns.

The terminal may treat some characters as special instructions. For SQL containing `$`, `!`, embedded quotes, or multiple lines, use a `.sql` file.

Copy commands from Markdown when possible. PDF line wraps, curly quotes, and copied prompt labels can alter a command. Enter one block at a time.

Practice: Seven Checks

1. In the shell: `duckdb --version` and the 42 checkpoint.
2. Confirm you are in this tutorial's folder.
3. In the shell: run the eight-row CSV checkpoint.
4. Run `duckdb`; at D, run the average-GPA query.
5. Try `.mode markdown`, then rerun the query.
6. Enter `.quit`; in the shell run `duckdb -bail -f report.sql`.
7. Create/reopen `class_cli.db` and verify its table persists.

If typing `duckdb` fails, use **Help Only If Needed** below.

First get the program working, then check the location of your data.

Help Only If Needed

If the practice worked, you have finished the lesson.

Use the following pages only for the problem you are seeing:

- `duckdb` is not found or not recognized.
- The program exists but cannot open.
- A CSV, SQL file, or saved table cannot be found.
- The database is locked, or the display looks unexpected.

Try the matching fix. If it does not help, use **Still Stuck?** at the end to send the instructor the details needed to help you.

Windows: “duckdb is not recognized”

In **either shell**:

```
where.exe duckdb  
duckdb --version
```

If not found: verify installation, restart the terminal application, and test the full executable path. Reopening alone cannot add a missing entry.

For WinGet, inspect `winget list --id DuckDB.cli --exact`.

For manual installation, verify the folder and User Path entry.

In PowerShell, `Get-Command duckdb -All` also reveals aliases or multiple copies. Bare `where` is a PowerShell alias, so use **where.exe**.

Windows: Continue Without Permanent PATH

For the manual installation, add the folder for **this session only**:

PowerShell:

```
$env:Path = "$env:USERPROFILE\duckdb-cli;$env:Path"  
duckdb --version
```

Command Prompt:

```
set "PATH=%USERPROFILE%\duckdb-cli;%PATH%"  
duckdb --version
```

Use the actual folder you installed into. These settings disappear when the shell closes; the full-path commands also work without changing PATH.

Windows: File Exists but Will Not Run

- **In its folder, PowerShell cannot find duckdb.exe:** use `.\duckdb.exe --version`; PowerShell does not search the current folder implicitly. Running it this way does not add it to PATH.
- **VCRUNTIME / MSVCP DLL missing:** install or repair the matching Microsoft Visual C++ Redistributable from Microsoft's site.
- **This app cannot run on your PC:** verify architecture, extract the Windows CLI zip, and check OS support. Do not download random DLLs.
- **Access denied / file disappears:** check device security notices; ask IT if execution or extraction is blocked.

Windows: SmartScreen and Managed PCs

If you see **Windows protected your PC**, confirm the executable came from DuckDB's official installation page.

Use **More info > Run anyway** only for a trusted download and when your device policy permits it. If that option is absent, contact IT.

Do not disable security software or assume WinGet bypasses policy.

For school PCs, a full-path command solves PATH lookup issues, but cannot solve a blocked executable or missing runtime dependency.

Recovery: CSV or SQL File Not Found

1. If at D, cancel unfinished input with Ctrl+C, then enter `.quit`.
2. In the shell, check the current folder and list the actual filename.
3. Change into the folder containing `data/`. For scripts, also check that you created `report.sql` there as instructed.
4. Retry the eight-row checkpoint, then `duckdb -bail -f report.sql`.

If using a synced folder (OneDrive/iCloud), download files locally first.

A full path can avoid ambiguity; use single quotes for paths in SQL.

If a path contains an apostrophe, double it inside the SQL string.

A `.csv` extension alone does not prove the file is valid CSV.

Recovery: Table, Lock, or Write Errors

- **Table does not exist:** you may be in a new in-memory session or the wrong database. Query the CSV directly or reopen the right `.db`.
- **Table already exists:** reuse it; do not drop it just to clear the error.
- **Could not set lock:** close other CLI sessions or Marimo/Python connections using that file, then retry. Do not delete lock/WAL files.
- **Read-only / cannot open:** check the parent folder, permissions, disk space, and whether you opened with `-readonly`.

Prefer a local writable folder for the database. Ask the instructor about version incompatibility errors; do not overwrite the existing database.

Recovery: Display or Startup Problems

- Garbled table borders? At D, try `.mode table` for ASCII borders. `.mode ascii` means a different machine-oriented format.
- Results look different? Check `.show`; reset with `.mode duckbox` and `.headers on`. A startup `.duckdbrc` can customize settings.
- To bypass it, first save a plain-text **clean_start.sql** containing only `-- No startup settings`. Then run in the shell:

```
duckdb -init clean_start.sql -c "SELECT 42 AS answer;"
```

Do not delete personal startup files. Different colors/layouts do not change query results.

Still Stuck? Send Useful Details

Send the instructor:

- Your OS and shell (Terminal/zsh, PowerShell, or Command Prompt).
- The exact command and complete error text.
- Whether a full-path `--version` command works.
- Your current folder and whether `data/students.csv` exists.
- `duckdb --version`, if available; redact private path information.

See [README.md](#) for copyable recovery commands and official references.

Examples use 1.5.5; check `.help` and `duckdb -help` for your installed version.